

Presentation: Build a Laravel Training System (with Bootstrap)

Slide 1: العنوان

Laravel + Bootstrap: Build a Training System Step by Step

- Laravel بناء نظام تدريب متكامل باستخدام
 - Bootstrap التصميم باستخدام
 - كامل للطلاب والمدرسين والكورسات والتسجيلات CRUD + تسجيل دخول وتسجيل مستخدم
-

Slide 2: فكرة المشروع

نظام إدارة التدريب:

- تسجيل دخول/تسجيل جديد للمستخدمين (طالب أو مدرس)
 - Enroll الطلاب يمكنهم استعراض الكورسات وعمل
 - المدرسون يمكنهم إنشاء كورسات
-

Slide 3: إنشاء Laravel Project + Breeze

```
composer create-project laravel/laravel training_system
cd training_system
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run build
```

Slide 4: إعداد قاعدة البيانات

 ملف `.env`:

```
DB_DATABASE=training_system
DB_USERNAME=root
DB_PASSWORD=
```

ثم:

```
php artisan migrate
```

Slide 5: إنشاء Models + Controllers + Migrations

```
php artisan make:model Student -mc  
php artisan make:model Course -mc  
php artisan make:model Teacher -mc  
php artisan make:model Enrollment -mc
```

Slide 6: وربط الجداول Migration تعديل

```
// database/migrations/create_enrollments_table.php  
$table->foreignId('student_id')->constrained()->onDelete('cascade');  
$table->foreignId('course_id')->constrained()->onDelete('cascade');  
$table->foreignId('teacher_id')->constrained()->onDelete('cascade');
```

ثم:

```
php artisan migrate
```

Slide 7: Models تعريف العلاقات بين

```
// Student.php  
public function enrollments() {  
    return $this->hasMany(Enrollment::class);  
}  
  
// Course.php  
public function enrollments() {  
    return $this->hasMany(Enrollment::class);  
}  
  
// Teacher.php  
public function courses() {  
    return $this->hasMany(Course::class);  
}
```

```
// Enrollment.php
public function student() {
    return $this->belongsTo(Student::class);
}
public function course() {
    return $this->belongsTo(Course::class);
}
public function teacher() {
    return $this->belongsTo(Teacher::class);
}
```

Slide 8: إنشاء Controllers

```
php artisan make:controller StudentController --resource
php artisan make:controller CourseController --resource
php artisan make:controller TeacherController --resource
php artisan make:controller EnrollmentController --resource
```

Slide 9: CRUD لتفعيل StudentController التعديلات الرئيسية في

```
use App\Models\Student;

public function index() {
    $students = Student::all();
    return view('students.index', compact('students'));
}

public function create() {
    return view('students.create');
}

public function store(Request $request) {
    $request->validate([
        'name' => 'required',
        'email' => 'required|email|unique:students'
    ]);
    Student::create($request->all());
    return redirect()->route('students.index');
}

public function edit(Student $student) {
    return view('students.edit', compact('student'));
}
```

```

}

public function update(Request $request, Student $student) {
    $request->validate([
        'name' => 'required',
        'email' => 'required|email|unique:students,email,' . $student->id
    ]);
    $student->update($request->all());
    return redirect()->route('students.index');
}

public function destroy(Student $student) {
    $student->delete();
    return redirect()->route('students.index');
}

```

Slide 10: للسماح فقط للمدرسين بالتعديل CourseController التعديلات على

```

public function create() {
    if (auth()->user()->role !== 'teacher') {
        abort(403);
    }
    return view('courses.create');
}

public function store(Request $request) {
    if (auth()->user()->role !== 'teacher') {
        abort(403);
    }
    // validation + creation logic
}

public function edit(Course $course) {
    if (auth()->user()->role !== 'teacher') {
        abort(403);
    }
    return view('courses.edit', compact('course'));
}

public function update(Request $request, Course $course) {
    if (auth()->user()->role !== 'teacher') {
        abort(403);
    }
    // validation + update logic
}

```

```
public function destroy(Course $course) {  
    if (auth()->user()->role !== 'teacher') {  
        abort(403);  
    }  
    $course->delete();  
    return redirect()->route('courses.index');  
}
```

Slide 11: تركيب Bootstrap

```
npm install bootstrap
```

```
resources/css/app.css:
```

```
@import 'bootstrap/dist/css/bootstrap.min.css';
```

```
resources/js/app.js:
```

```
import 'bootstrap';
```

ثم:

```
npm run build
```

Slide 12: ملف الرئيسي layout app.blade.php

```
<!-- resources/views/layouts/app.blade.php -->  
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>{{ config('app.name', 'Laravel') }}</title>  
    @vite(['resources/css/app.css', 'resources/js/app.js'])  
</head>  
<body>  
    <nav class="navbar navbar-expand-lg navbar-light bg-light">
```

```

<div class="container">
  <a class="navbar-brand" href="/">Training System</a>
  <div class="collapse navbar-collapse">
    <ul class="navbar-nav ms-auto">
      @auth
        <li class="nav-item">
          <a class="nav-link" href="#">{{ auth()->user()-
>name }}</a>
        </li>
        <li class="nav-item">
          <form method="POST" action="{{ route('logout') }}">
            @csrf
            <button class="nav-link btn btn-link"
type="submit">Logout</button>
          </form>
        </li>
      @else
        <li class="nav-item"><a class="nav-link"
href="{{ route('login') }}">Login</a></li>
        <li class="nav-item"><a class="nav-link"
href="{{ route('register') }}">Register</a></li>
      @endauth
    </ul>
  </div>
</div>
</nav>
<main class="container mt-4">
  @yield('content')
</main>
</body>
</html>

```

Slide 13: صفحات الطلاب (Students CRUD)

(كما هو مذكور سابقاً) ...

Slide 14: صفحات الكورسات (Courses CRUD)

(كما هو مذكور سابقاً) ...

الخطوة التالية؟ Slide 15:

الأساسي CRUD بعد الانتهاء من .

- Roles (Teacher, Student) إضافة صلاحيات
- عرض الكورسات فقط للطلاب
- عرض الطلاب المسجلين فقط للمدرسين
- منفصلة لكل نوع مستخدم Dashboard تصميم صفحة